DA2304: Introduction to Computer Systems

Assignment 2: Virtual Machines

Data Science and AI Department (DSAI)
Indian Institute of Technology Madras

`Instructor`: Dr. Patanjali SLPSK (`patanjali@dsai.iitm.ac.in`)

Release Date: April 20, 2026
Deadline: April 27, 2026, 11:59 PM

Introduction

In Assignment 1, you dealt with hardware implementations and the raw Hack instruction set. In **Part 1** of this assignment, you will step up the abstraction ladder to the Virtual Machine (VM) paradigm. You will build a **VM Translator** in Python that compiles stack-based VM commands into native Hack assembly.

In **Part 2**, you will write high-level algorithmic logic—specifically, Matrix Multiply-Accumulate (MAC) operations ($W \cdot X + B$)—and analyze how different function-calling strategies impact the stack, memory segments, and overall execution efficiency on our architecture.

Prerequisites

1. Understanding of Stack-based Virtual Machine architecture.
2. Proficiency in Python to build a two-pass parser/translator.
3. Familiarity with Hack Assembly Language and its memory segments (`local`, `argument`, `this`, `that`, `pointer`, `temp`).
4. Linear algebra basics (Matrix Multiplication).

Part 1: The VM Translator

1 Exercise 1: Building the Hack VM Translator

Your task is to write a Python program that translates `.vm` files into `.asm` files. The resulting assembly must execute correctly on the original Hack HDL CPU instruction set.

Requirements:

1. **Stack Arithmetic:** Implement commands like `add`, `sub`, `neg`, `eq`, `gt`, `lt`, `and`, `or`, `not`.
2. **Memory Access:** Implement `push` and `pop` across all 8 memory segments.
3. **Program Flow & Function Calls:** Implement `label`, `goto`, `if-goto`, `function`, `call`, and `return`.
4. **Integration:** Create a `HackVM` module. Load it into your main script, process standard `.vm` test files provided in the course repository, and verify the resulting machine instructions.
5. The test files are the `.vm` files in the Nand2Tetris Repo (Project 07/08) as well as in our institute hosted HACK HDL Simulator's VM Tab.

Part 2: Matrix Operations (MAC)

1 Exercise 1: The MAC Operation Architecture

In modern AI workloads, the Multiply-Accumulate (MAC) operation is the foundational building block. You will implement a MAC operation of the form:

$$Y = W \cdot X + B$$

where W and X are input matrices, and B is a bias matrix (a column vector).

1.1 Sanity Checking

Before initiating any matrix multiplication, you must perform a sanity check to verify that the matrices are compatible.

`HINT:` You can support the *throw_error*() functionality by setting and resetting a particular memory location to -1 or 1.

Question: Write a brief pseudocode snippet to validate the dimensions of W, X, and B before execution.

1.2 Approach A: The `MATMUL` Function Call

In this approach, the function `MATMUL(W_ptr, X_ptr)` is called *once*. It takes pointers to the entire matrices and their dimensions as arguments. It contains nested loops internally, performs the complete multiplication, and a separate function that adds B, and returns a pointer to the resulting matrix Y. B will be same as $W \times X$.

1.3 Approach B: The `ROWXCOL` Function Call

In this alternative approach, the caller manages the nested loops. The inner loop repeatedly calls a subroutine `ROWXCOL(row_ptr, col_ptr, length)`, which calculates the dot product for a single entry of the output matrix.

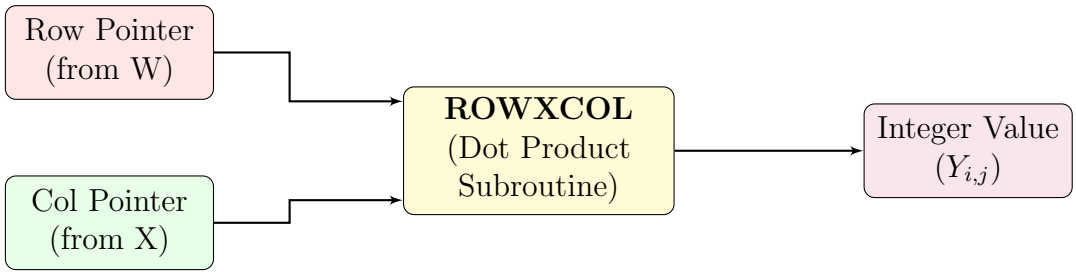

Figure 1: Data flow for a single `ROWXCOL` function call.

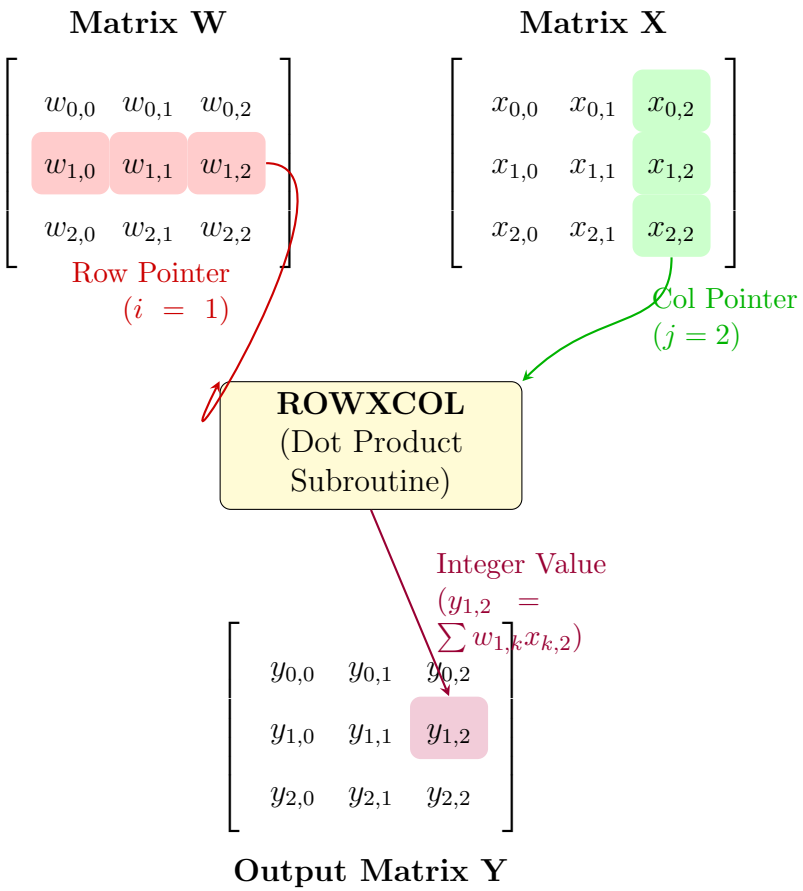

Figure 2: Data flow for a single `ROWXCOL` call computing element $(1, 2)$ of a 3×3 matrix.

2 Exercise 2: Architectural Analysis

Based on the Hack VM architecture, analyze Approach A (`MATMUL`) and Approach B (`ROWXCOL`).

1. **Stack Usage:** Comment on the stack usage and function call overhead in Approach A vs. Approach B.
2. **Segment Usage:** How do these two approaches differ in their utilization of the various memory segments?
3. **Optimization (Approach A):** What specific hardware or software-level optimizations can be applied to Approach A to speed up execution?
4. **Optimization (Approach B):** What compiler-level optimizations can be applied to Approach B to mitigate its inherent architectural disadvantages?

Submission Instructions

1. Theoretical answers, analysis, and sanity check logic (Part 2) should be in a report titled `Name_RollNo.pdf`.
2. **Software:** Place your Python VM Translator code in a folder named `src/`. Ensure there is a `main.py` and a `README.md` detailing how to execute the translator.
3. Include at least three example `.vm` programs (including your Matrix MAC implementations) that you successfully translated and tested.
4. Compress the PDF, `.vm` files, and `src/` folder into a **ZIP archive** named: `DA2304_Assignment2_RollNumber.zip`
5. Upload the archive to the course portal by April 27, 2026, 11:59 PM.
6. **Report Extra Credit:** In the report, add a separate section titled "Extra Credit", and answer the following question: What is your recent favorite song? (2 points)

Good luck! For queries, post on the course forum or contact the TAs.